

BACKSTAGE

Documento de Contexto e Arquitetura do Projeto

1. Origem do Projeto

O projeto Backstage surgiu da necessidade de digitalizar e organizar a gestão operacional de eventos, identificada a partir de uma planilha real utilizada no evento Boombay 2026 (18 a 26 de abril de 2026). A planilha continha escalas de equipes divididas em: Carregadores, Limpeza, Lojinha/Guarda Volumes, Caixas, Bilheteria e controle de Veículos.

2. Objetivo

Criar um sistema web em Ruby on Rails para gerenciar múltiplos eventos, com controle de equipes, setores, escalas de colaboradores, perfis de acesso e veículos.

3. Stack Tecnológica

Tecnologia	Finalidade
Ruby on Rails	Framework principal (back-end e front-end)
PostgreSQL	Banco de dados relacional
Devise	Autenticação de usuários
Pundit	Autorização baseada em perfis (RBAC)
Hotwire (Turbo + Stimulus)	Interatividade sem APIs separadas
Bootstrap	Framework CSS para interface
Docker + Docker Compose	Ambiente de desenvolvimento containerizado

Estrutura Docker:

```
docker-compose.yml
```

```
■■■ web      (Rails app)
```

```
■■■ db      (PostgreSQL)
```

```
■■■ redis   (opcional – Sidekiq/ActionCable futuramente)
```

4. Modelo de Dados

4.1 Colaborador (Usuário)

Entidade central do sistema. Todo colaborador tem login e perfil de acesso.

```
nome, cpf, telefone, email, senha (Devise), perfil_id
```

4.2 Perfil e Permissões

O sistema utiliza RBAC (Role-Based Access Control) com Pundit. As permissões são customizáveis pelo admin via interface — cada perfil define quais recursos pode acessar e quais ações pode executar (criar / editar / visualizar / deletar).

```
Perfil: nome
```

```
Permissao: perfil_id, recurso, acao
```

4.3 Evento

Entidade principal que agrupa toda a operação. O status controla o ciclo de vida do evento.

```
nome, local, data_inicio, data_fim
```

```
status: rascunho | em_andamento | finalizado
```

4.4 Hierarquia Evento → Escala

```
Evento
```

```
■■■ Equipe (nome)
```

```
■■■ Setor (nome – todas as equipes têm setor)
```

```
■■■ Escala
```

```
■■■ colaborador_id
```

```
■■■ data, hora_inicio, hora_fim
```

```
■■■ tem_radio (boolean)
```

4.5 Veículo

Entidade independente. Pode estar vinculado a um colaborador e opcionalmente alocado a um evento específico.

```
colaborador_id, evento_id (opcional), modelo, cor, placa
```

5. Regras de Negócio

1. Um colaborador pode trabalhar em múltiplos eventos, desde que não haja conflito de horário.
2. O sistema valida conflito de horário antes de confirmar uma escala.

3. O CPF é o identificador único do colaborador no sistema.
4. Toda equipe obrigatoriamente possui pelo menos um setor.
5. A escala sempre é vinculada a um setor (nunca diretamente à equipe).
6. O status do evento segue o fluxo: rascunho → em_andamento → finalizado.
7. As permissões de cada perfil são definidas e customizadas pelo administrador via interface.

6. Próximos Passos

A geração do projeto seguirá a ordem abaixo, partindo da base que todos os módulos dependem:

1. Setup do projeto Rails + Docker + Docker Compose
2. Configuração do Devise (autenticação de Colaboradores)
3. Cadastro de Perfis e Permissões (Pundit + interface admin)
4. CRUD de Colaboradores
5. CRUD de Eventos
6. CRUD de Equipes e Setores
7. Gestão de Escalas (com validação de conflito de horário)
8. Gestão de Veículos